

Deploy di WeatherApp su GCP

Università degli Studi di Cagliari

Corso di Cloud Infrastructures and Security

Cocco Mattia 65336

1. Risorse

Questo Progetto tratta il deploy dell'applicazione sviluppata durante il corso di Cloud Infrastructures and Security (**WeatherApp**) su Google Cloud Platform.

Il codice è disponibile nel **repository**: <https://github.com/Matti02co/cis-project-2526> .

Un **video** illustrativo è disponibile al link:

https://drive.google.com/file/d/1lpbNdqQfCMK2B7XjMxZVC3F2tCt_80HE/view?usp=sharing

2. Introduzione

GCP è una suite completa di servizi di cloud computing basata sull'infrastruttura globale di Google, che offre risorse di calcolo e archiviazione, ambienti serverless, servizi di PaaS, IaaS e altro, per permettere la creazione di applicazioni e servizi su larga scala, garantendo scalabilità, sicurezza e affidabilità.

WeatherApp è un'applicazione web che consente di visualizzare le previsioni meteo delle località inserite in input e si basa su container docker che comunicano tramite una rete bridge.

Per eseguire il deploy dell'applicazione sulla piattaforma cloud sono state utilizzate le immagini fornite a laboratorio e la configurazione del proxy utilizzata per AWS, adattate come verrà spiegato in seguito.

La fase preliminare ha interessato la creazione di un account su GCP e il collegamento di un metodo di pagamento, permettendo di usufruire dei **300\$** di prova. In seguito, è stato possibile creare un progetto collegato a questo account di fatturazione, chiamato **cis-project2526**:

Progetti collegati a questo account di fatturazione

Nome progetto	ID progetto	Azioni ?
My First Project	project-ab57cd72-b212-4248-b11	⋮
cis-project2526	cis-project2526	⋮

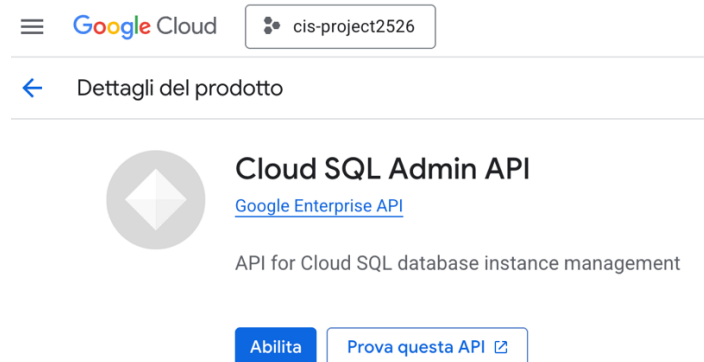
3. Setup delle Istanze

Dopo aver creato il progetto, sono state aggiunte due istanze (seguendo un po' la logica utilizzata per AWS):

- Cloud SQL
- Compute Engine

3.1. Cloud SQL

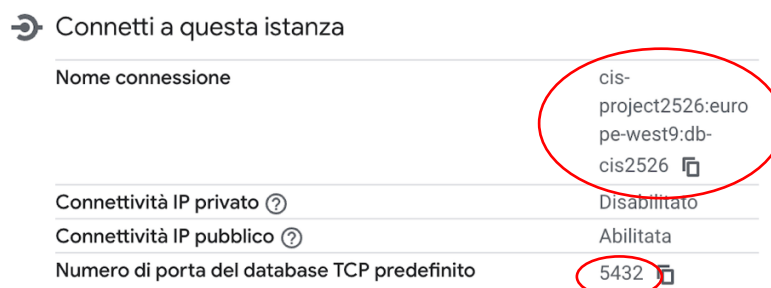
L'istanza di Cloud SQL è stata utilizzata per il **database**, in seguito all'abilitazione delle relative API:



Queste sono le opzioni scelte durante la configurazione:

- **Preset:** Sviluppo
- **Versione:** PostgreSQL 15
- **Zona:** europe-west9, Parigi (Singola per motivi di budget)

A questo punto era già possibile vedere il nome della connessione e la porta che si sarebbero utilizzate per connettersi all'istanza:



Infine, sono stati aggiunti un **database** (cisdb) e un **utente** (cisuser), avendo cura di salvare la **password** per il futuro accesso al database

3.2. Compute Engine

L'istanza di Compute Engine consiste in una **Virtual Machine**, sulla quale verranno eseguiti i container di WeatherApp. Anche in questo caso è stato necessario abilitare le opportune API:



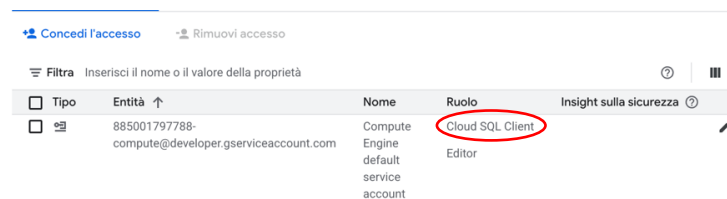
Le opzioni scelte durante la configurazione sono:

- **Regione:** europe-west9 (Parigi)
- **Macchina:** E2 (per motivi di budget. Sarebbe stata meglio una macchina flessibile)
- **Dimensioni:** 20GB (budget)
- **Immagine:** Ubuntu 22.04 LTS

Alcune delle operazioni più importanti sono state effettuate sul **firewall**, in particolare:

- È stato abilitato **HTTP**
- È stata inserita una regola per consentire il **traffico** in entrata da parte di tutti gli indirizzi IP.

Infine, per poter utilizzare con la VM il database creato in precedenza, è stato fornito l'accesso alle API al **service account** di Compute Engine, a cui è stato aggiunto il ruolo di **Cloud SQL Client**:



Identity and API access ⓘ

Service account ⓘ

Service account

Compute Engine default service account

Per accedere alle istanze con questo service account, devi aggiungere il ruolo Service Account User: roles/iam.serviceAccountUser. [Scopri di più](#)

Ambiti di accesso ⓘ

☐ Consenti l'accesso predefinito

☒ Consenti l'accesso completo a tutte le API Cloud

☐ Imposta l'accesso per ogni API

A questo punto era già possibile vedere l'**indirizzo IP esterno** della VM, tramite il quale sarebbe stata accessibile l'applicazione:

età

ack IP	Indirizzo IP esterno	Live
	34.155.162.233 (Temporaneo) 	Prei

4. Modifiche della configurazione docker

Il **docker-compose** utilizzato è simile a quello fornito per AWS, con le componenti originali di WeatherApp e il proxy. È stato però aggiunto un servizio: il **Cloud SQL Proxy**, per consentire all'applicazione di accedere direttamente all'istanza di Cloud SQL che contiene il database:

```
cloudsql-proxy:
  image: gcr.io/cloud-sql-connectors/cloud-sql-proxy:2.8.0
  command:
    - "--structured-logs"
    - "--address=0.0.0.0"
    - "--port=5432"
    - "${CLOUDSQL_INSTANCE}"
  restart: always
```

La variabile **CLOUDSQL_INSTANCE** deve contenere il nome della connessione all'istanza di Cloud SQL, che è stato mostrato nella sezione precedente. Anche la **porta** deve corrispondere a quella predefinita per il database, osservata precedentemente nelle opzioni di connessione

Inoltre, dato che Strapi provava a connettersi con **SSL** di default, ma Cloud SQL gestisce internamente la sicurezza e non lo supporta direttamente, si è deciso di modificare l'immagine del cms, aggiungendo un volume che monta un file **database.js** personalizzato:

```
app-network
volumes:
- ./database.js:/opt/app/config/database.js
```

L'unica modifica all'interno del file riguarda appunto SSL per postgres:

```
ssl: env.bool("DATABASE_SSL", false)
? {
  rejectUnauthorized: true,
  ca: process.env.DATABASE_SSL_CA
  ? fs.readFileSync(process.env.DATABASE_SSL_CA)
  : undefined,
}
: false,
schema: env("DATABASE_SCHEMA", "public"),
},
```

4.1. Variabili d'ambiente

Dopo la modifica del docker-compose originale, i file sono stati caricati sull'istanza di Compute Engine tramite Github ed è stato creato il file **.env** per le variabili d'ambiente, che contiene alcune informazioni fondamentali:

- **L'istanza Cloud SQL**, contenente il nome della connessione per il database (visto all'inizio)
- **Nome utente, nome database e password** per l'accesso (mostrati nella sezione 3.1)
- **Chiave** per le API del meteo
- Altri **segreti** utili per Strapi

```
CLOUDSQL_INSTANCE=cis-project2526:eu-west-9:db-cis2526
DB_NAME=cisdb
DB_USER=cisuser
DB_PASSWORD=Password-cis2526

WEATHER_API_KEY=c2cc615e2ab5b48bdf9a0672c434a819

JWT_SECRET=secret
APP_KEYS=u7jiX+ECXgikHyYBN4qY3w==,ixMJavk01LEXelPpvOhJ2w==
API_TOKEN_SALT=BGkd9pPHiUmGCPNsFY8+wQ==
ADMIN_JWT_SECRET=uCOPM9KH+E9Mb5kArk1Z+Q==
TRANSFER_TOKEN_SALT=wFFCWK3brgh1/CsH8PMGxQ==
```

Una volta fatto ciò, è stato utilizzato docker per eseguire l'applicazione.

Conclusioni

Dopo aver completato tutti i passaggi precedenti, con il database collegato e l'applicazione in esecuzione sulla VM, è possibile accedervi tramite l'IP pubblico dell'istanza di Compute Engine (mostrato nella sezione 3.2). Si possono creare gli utenti accedendo a Strapi da **<IP>/admin** e utilizzare le funzioni dell'applicazione su **<IP>/fe**, come viene mostrato nel video raggiungibile dalla sezione 1. Risorse.

